C 프로그래밍

학습목표

- C 언어로 프로그램을 작성할 수 있다.
- 컴퓨팅 사고력을 증진시킨다.

개요

C언어는 현대 대부분의 프로그래밍 언어의 근간이 되는 언어로 절차지향 프로그래밍과 구조적 프로그래밍을 지원하는 대표적인 언어다. C 언어를 배우며 프로그래밍의 기초를 다지도록 한다.

기본 개념

우리는 컴퓨터에게 어떻게 명령을 내릴 수 있을까? 프로그램은 어떻게 실행이 되는 걸까? 문법에 들어가기에 앞서 몇 가지 기본 개념을 살펴보자.

식별자

식별자(Identifier)는 프로그램에서 여러 요소를 구분하는 데 사용된다. 식별자는 프로그램에서 특수한 의미를 가지는 키워드(Keyword)를 제외하고 일련의 규칙에 맞춰 작성할 수 있다. 규칙은 아래와 같다.

- 숫자로 시작할 수 없다.
- 대소문자를 구별한다.

물론 규칙에 맞춰 적었다고 장땡은 아니다. 모든 식별자는 항상 구체적으로 적어야 가독성이 좋다.*

* 이를 위해 코딩 컨벤션(Coding Convention)이라는 것이 있다.

```
// 나쁨
t = s + l - b;

// 좋음
TotalLeaves = SmallLeaves + LargeLeaves - SmallAndLargeLeaves;
```

선언

식별자는 **선언**(Declaration)을 통해 프로그램에게 알려진다. 선언을 통해 식별자의 의미와 속성을 지정해줄 수 있다. 특히 식별자에 대한 모든 정보를 제공하는 선언을 **정의**(Definition)라 한다. 식별자는 선언될 때 **범위**(Scope)를 가지며, 네 개의 **이름 공간**(Name Space) 중 하나에 속하게 된다. 어떤 식별자는 서로 다른 범위나 번역 단위를 가질 때, 그것을 참조할 수 있는 **연결**(Linkage)을 가진다.

범위는 프로그램이 식별자를 볼 수 있는(Visible) 영역이다. 범위는 네 가지가 있다.

- 블록 범위(Block Scope)
  - 복합 구문 안 쪽에서만 식별자를 볼 수 있는 영역이다.
  - 블록 범위 안 쪽에는 또다른 블록 범위가 있을 수 있다.
- 파일 범위(File Scope)
  - 파일 전체에서 식별자를 볼 수 있는 영역이다.
- 함수 범위(Function Scope)
  - 함수 내 레이블(Label)이 갖는 영역이다.
- 함수 프로토타입 범위(Function Prototype Scope)
  - 함수 선언에서 매개변수 목록에 소개된 식별자의 영역이다.

C 프로그램은 식별자에 카테고리를 지정해 같은 식별자라 할지라도 구분할 수 있다. 이런 카테고리를 이름 공간이라고 한다. C에서 이름 공간은 4가지가 있다.

- 레이블 이름 공간
  - 레이블만 저장한다.
- 태그 이름 공간
  - 구조체, 공용체, 열거형의 이름을 저장한다.
- 멤버 이름 공간
  - 구조체나 공용체의 멤버 이름을 저장한다. 모든 구조체와 공용체는 독자적인 공간을 갖고 있다.
- 일반 이름 공간
  - 위의 세 가지를 제외한 나머지의 이름을 저장한다.

구문

구문(Statement)은 컴퓨터에게 명령을 내리기 위해 사용한다. 구문에는 5가지 종류가 있다.

- 복합 구문(Compound Statement)
  - 중괄호를 통해 여러 구문을 적은 것을 의미한다.
  - **블록**(Block)이라고도 한다.
- 식 구문(Expression Statement)
  - 계산을 정하는 **연산자**(Operator)와 **피연산자**(Operand)의 시퀀스다.

    - 계산을 하는 것을 **평가**(Evaluation)라 한다.
        - 평가는 결과를 만들거나, 사이드 이펙트(Side-Effect)를 생성하거나 변수나 함수를 지정할 수 있다.
    - 아무것도 적혀 있지 않은 식 구문을 **널 구문**(Null Statement)이라고 한다.
- 분기문(Selection Statement)
    - 식의 계산에 따라 몇 개의 구문 중 하나를 선택하게 하는 구문이다.
    - 프로그램에 여러 흐름을 만든다.
    - if / switch가 있다.
- 반복문(Iteration Statement)
    - 특정 구문을 반복하는 구문이다.
    - while / do-while / for가 있다.
- 점프문(Jump Statement)
    - 조건에 관계없이 흐름을 바꾸는 구문이다.
    - break / continue / return / goto가 있다.

주석

주석(Comment)은 사용자에게 정보를 남기는 문서 역할을 한다. 프로그램에는 아무 효용이 없으며 오직 코드를 읽는 사람에게만 효과가 있다. 주석에는 2가지가 있다.

```
/*
여러줄을 작성할 수 있다.
*/

// 한 줄만 작성할 수 있다.
```

좋은 주석을 남기는 방법은 여기를 참고하자.

Main 함수

이제까지의 내용을 정리해보자면 우리는 구문을 통해 컴퓨터에게 명령을 내릴 수 있으며, 구문에 사용되는 여러 프로그램의 요소는 식별자에 의해 구분된다는 것을 알 수 있다. 이렇게 작성된 것은 Main 함수에서 시작이 되며, 이를 프로그램의 **진입점**(Entry Point)이라 한다. 아래는 가장 간단한 (그리고 쓸모없는) 프로그램이다.

```
int main(void)
{
    return 0;
```

```
}
```

객체

문제 해결을 위해선 데이터를 메모리에 저장하고 이를 조작해야 한다. 이렇게 데이터가 저장되는 영역을 객체(Object)라 한다. 객체는 선언을 통해 생성될 수 있고, 모든 객체는 아래와 같은 속성을 가진다.

- **크기**(Size)
  - 데이터를 저장하기 위한 바이트의 개수를 의미한다.
  - sizeof 연산자를 사용해 구할 수 있다.
- 정렬 요건(Alignment Requirement)
  - 데이터를 저장하기 위해 필요한 연속적인 바이트 개수다.
- 기억 존속 시간(Storage Duration)
  - 데이터를 얼마나 오래 가질 수 있는지를 의미한다.
- **수명**(Lifetime)
  - 객체가 유효한 시간을 의미하며 기억 존속 시간과 같다.
  - 블록 범위를 갖는 객체의 수명은 정의된 순간부터 블록 끝까지이다.
  - 파일 범위를 갖는 객체의 수명은 프로그램 시작부터 프로그램 끝날 때까지이다.
- **타입**(Type)
  - 객체가 가지고 있는 값의 해석 방법이다.
- **값**(Value)
  - 데이터를 의미한다.
- **식별자**(Identifier)
  - 객체의 이름을 의미한다.

먼저 기본 타입의 객체에 대해서 알아가보자.

정수

C 프로그램에서 정수는 부호가 있는 정수와 부호가 없는 정수로 나뉜다.

```
signed char obj1; // 크기는 최소 1바이트
short obj2; // 크기는 최소 2바이트
int obj3; // 크기는 최소 2바이트
long obj4; // 크기는 최소 4바이트
long long obj5; // 크기는 최소 8바이트
```

위의 타입은 부호를 가지고 있다. MSB(Most Significant Bit)를 부호 비트로 사용해 0이면 양수, 1이면 음수로 취급한다. 음수는 2의 보수를 이용하여 표현한다. 2의 보수는 1의 보수에 +1을 해주면 된다. -12을 2진수로 표현해보자면 아래와 같다.

1. 먼저 12을 이진수로 표현한다. ⇒ 0000 1100
2. 1의 보수를 취해준다. ⇒ 1111 0011
3. +1을 더해준다 ⇒ 1111 0100

만일 부호 없이 사용하고 싶다면 unsigned를 붙이면 된다.

```
unsigned char obj1;
unsigned short obj2;
unsigned int obj3;
unsigned long obj4;
unsigned long long obj5;
```

객체를 정의할 때 초기값을 지정할 수 있다. 이런 과정을 초기화(Initialization)라고 한다. 정수 객체에 데이터를 저장하기 위해서 정수 상수(Integer Constant)를 사용할 수 있다. 정수 상수에도 타입이 있으며 정수 상수의 타입은 접미사를 이용한다. 아래를 보자.

```
int number = 42; // int 타입의 42
unsigned int number2 = 42U; // unsigned int 타입의 42
long number3 = 42L; // long 타입의 42
unsigned long number4 = 42UL; // unsigned long 타입의 42
long long number5 = 42LL; // long long 타입의 42
unsigned long long number6 = 42ULL; // unsgined long long 타입의 42
```

정수 상수는 접두사를 이용하면 10진법 외에도 8진법이나 16진법으로 나타낼 수 있다. 큰 수를 나타내거나 이진수를 다룰 때 유용하다.

```
number = 42; // 10진수
number = 052; // 8진수
number = 0x2a; // 16진수
number = 0X2a; // 16진수
```

정수 타입에 사용할 수 있는 연산자는 기본적으로 산술 연산(Arithmetic Operation)이 있다.

```
signed char number = 15;
```

```
+number; // 15
-number; // -15
number + 20 // 35
number - 10 // 5
number * 2 // 30
number / 10 // 1
number % 10 // 5

~number; // 0xF0
number & 0x03; // 0x03
number | 0x12; // 0x1F
number ^ 0x3F; // 0x30
number << 1; // 0x1E;
number >> 1; // 0x07;
~number >> 1; // 0xF8;
```

이외에 증감 연산(Increment/Decrement Operation)이 있다.

```
int a = 1;
int b = a++; // b = 1, a = 2
int c = ++a; // c = 3, a = 3
```

실수

C 언어에서는 실수를 표현하기 위해 부동 소수점으로 표현하고, 그 중 IEEE-754 포맷을 사용한다. IEEE-754는 비트를 부호, 지수, 정규화된 가수 부분으로 나눠 표현하는 방식이다. 실수 이외에도 무한대, NaN 등의 특수 값이 지정되어 있다.

```
float obj1; // float의 크기는 4바이트
double obj2; // double의 크기는 8바이트
long double obj3; // long double의 크기는 8바이트 혹은 16바이트
```

정수 상수와 마찬가지로 부동 소수점 상수(Floating Constants)가 있으며, 접미사를 통해 타입을 구분지을 수 있다.

```
float realNumber = 1.2f; // float 타입의 1.2
double realNumber2 = 1.2; // double 타입의 1.2
long double realNumber3 = 1.2L; // long double 타입의 1.2
```

또, 과학적 표기법이 가능하다.

```
realNumber = 123.456;
realNumber = 1.23456e2; // 1.23456 * 10^2
```

부동 소수점 타입에는 모듈로 연산자를 제외한 산술 연산과 증감 연산이 가능하다.

문자

문자는 char 타입을 사용하며, ASCII 코드로 해석한다. 위 타입과 마찬가지로 문자 상수(Character Constants)가 있다. char 타입은 정수 타입이기도 하므로 산술 연산 및 증감 연산이 가능하다.

```
char ch = 'A';
char ch2 = '\n';
char ch3 = '\13';
```

변수와 상수

지금까지 산술 타입(Arithmetic Types)에 대해 알아봤다. 지금까지 우리가 만든 객체를 **변수**(Variable)라고 하는데, 이는 객체에 저장된 값을 수정할 수 있다는 의미를 갖고 있다. 예시를 보자.

```
int main(void)
{
    int n = 10; // int 타입의 객체 n을 10으로 초기화했다.
                // 이때 n의 값은 10이다.
    n = 20; // n의 값을 20으로 할당(assignment)했다.
            // 이처럼 객체에 값을 할당해주는 연산자를 할당 연산자라고 한다.
            // 이때 n의 값은 20이다.
    ++n; // n의 값을 증가시켰다.
         // 이때 n의 값은 21이다.

    int a = n; // int 타입의 객체 a를 n의 값으로 초기화했다.
               // 이때 a의 값은 21이다.

    return 0;
}
```

때로는 값이 수정되지 않길 원할 때도 있을 것이다. 그럴 때 const 타입 수식자(Type Qualifier)를 써주면 정의된 이후 값이 수정되지 않도록 컴파일 단계에서 막을 수 있다. 이렇게 데이터를 변경할 수 없는 객체를 **상수**(Constant)라 한다.

```
int main(void)
{
    const int a = 10;
    a = 20; // 컴파일 오류(Compile Error)

    int const b = 20; // 타입 수식자는 타입 뒤에도 붙일 수 있다.

    return 0;
}
```

보통 상수를 만드는 것은 어떤 값에 이름을 붙이기 위해서이다. 아래의 값을 보자.

```
299792458
```

위의 값이 무얼 지칭하는지 알 수 없다. 하지만 아래와 같이 쓴다면 어떨까?

```
const int SPEED_OF_LIGHT = 299792458;
```

이제는 저 값이 빛의 속도를 의미한다는 것을 알 수 있다. 이런 것처럼 사람은 숫자와 친하지 않고 문자와 친하다. 위에 적었던 299792458처럼 어떤 의미를 가지는지 모르는 값을 **마법의 숫자**(Magic Number)라고 하며, 이는 코드의 가독성을 매우 떨어뜨린다. 따라서 되도록 마법의 숫자를 적는 대신 이름을 붙여 가독성을 높이도록 하자. 이를 상수에 이름을 붙였다 하여 **기호 상수**(Symbolic Constant)라 한다.

변환

때로는 데이터를 다른 타입으로 **변환**(Conversion)해야 할 때도 있다. 가령 정수형 데이터를 가지고 실수를 얻어내야 할 때 그렇다. 변환에는 암시적 변환(Implicit Conversion)과 **명시적 변환**(Explicit Conversion)이 있다.

암시적 변환은 다른 타입의 값이 예상될 때 일어난다. 아래의 사례를 보자.

```
int n = 1L; // 값은 long 타입이지만 n의 타입은 int다.
```

```
int n2 = 2.1; // 값은 double 타입이지만 n2의 타입은 int다.
double d = 2; // 값은 int 타입이지만 d의 타입은 double이다.
const int c = n2; // 값은 int 타입이지만 c의 타입은 const int다.
```

우리가 지금까지 배운 것 중에서 변환이 일어날 때는 아래와 같다.

- 할당이나 초기화를 할 때, 오른쪽 피연산자의 타입이 왼쪽 피연산자의 타입으로 변환된다.
- 산술 연산을 할 때 아래와 같은 규칙으로 일어난다.
    - 부동 소수점 타입이 있다면 부동 소수점 타입으로 변환된다.
    - 부동 소수점끼리 연산을 한다면 둘 중 크기가 큰 쪽으로 변환된다.
    - 정수끼리 연산을 한다면 정수 승격(Integer Promotion)이 일어난다.
        - 서로 크기가 다르다면 큰 쪽으로 변환된다.
        - 크기가 같을 때 unsigned 타입이 있는 경우 unsigned으로 변환된다.

명시적 변환은 캐스팅(Casting)이라고 한다. 아래의 예시를 보자.

```
double result = 1 / 3; // 우리가 예상한 것은 result에
                       // 0.3333333이라는 값이 저장되는 것이지만
                       // 실제로는 0.0이 저장된다.
                       // 그 이유는 1과 3의 타입이 모두 int이고,
                       // 따라서 / 결과 또한 int이기 때문이다.
                       // 그러므로 1 / 3의 몫인 0이 결과로 나오게 된다.
                       // 이후, double 타입으로 암시적 변환이 일어난다.
                       // 즉, 0.0이다.
```

이 경우 캐스트 연산자를 사용해 변환을 해주면 된다.

```
double result = 1 / (double)3; // int 타입의 값 3을 double 타입으로
                               // 명시적 변환을 했다.
                               // 따라서 (double)3은 3.0이 된다.
                               // 이후 / 연산자의 피연산자가 서로 타입이
                               // 다르므로 암시적 변환이 일어난다.
                               // 즉, int 타입의 값 1은
                               // double 타입의 값 1.0으로 변환된다.
                               // 최종적으로 1.0 / 3.0이므로
                               // 결과값은 0.333333이 된다.
```

캐스팅 역시 규칙이 있어 멋대로 변환을 할 수 있는 것은 아니지만 정수나 실수, 문자끼리는 자유롭게 변환할 수 있다는 것만 알아두자. 이후 규칙은 차차 알아갈 것이다. 또, 변환 시에

주의해야 할 점은 데이터가 손실 될 수 있다는 것이다. 항상 식의 타입이 무엇인지 인지하고, 데이터 손실이 일어날 수 있는지 확인하도록 하자.

표준 입출력

이제까지 객체를 통해 데이터를 저장할 수 있다는 것을 배웠다. 이번에는 입출력에 대해서 배워보도록 하자. 입출력은 표준 입출력 라이브러리*인 <stdio.h>를 사용한다. 이중 형식화된 입출력을 할 수 있는 scanf()와 printf()를 사용한다.

*라이브러리(Libarary)는 여러 유용한 함수를 모아놓은 걸 말한다.

```
// .h 파일을 헤더 파일(Header File)이라고 부르며
// 여러가지 선언이 적혀 있는 파일이다.
// #include는 전처리기(Preprocessor)로 다른 파일을 현재 파일에 '포함'시킨다.
#include <stdio.h>

int main(void)
{
    printf("정수를 입력해주세요.\n");
    int n;
    scanf("%d", &n);
    // & 연산자는 주소 연산자(Address Operator)로 객체의 주소를 계산한다.
    printf("당신이 입력한 값 : %d", n);

    printf("실수를 입력해주세요.\n");
    float f;
    scanf("%f", &f);
    printf("당신이 입력한 값 : %f", f);

    printf("문자를 입력해주세요.\n");
    char ch;
    scanf("%c", &ch);
    printf("당신이 입력한 값 : %c", ch);

    return 0;
}
```

문제를 풀어보자.

분기문

지금까지 작성한 프로그램은 모두 선형적이었다. 다시 말해 프로그램의 실행 흐름이 일정하며 다양한 동작이 없다. 이번 파트에서는 프로그램의 실행 흐름에 분기를 발생시켜 여러 흐름을 만들 수 있는 분기문에 대해서 배워보자.

진리값

분기문을 다루기 전 **진리값**(Truth Value)에 대해서 알아야 한다. 진리값은 명제의 내용이 참인지 거짓인지를 나타내는 값을 의미한다. 참과 거짓 판별은 값에 의해 결정되는 데, 0일 경우 거짓, 0 이외의 값은 모두 참으로 판단한다. 각 타입에 따라 진리값을 나타내면 아래와 같다.

```
// 정수
0; // 거짓
1; // 참
-1; // 참

// 실수
0.0; // 거짓
1.2; // 참

// 문자
'a'; // 참
'\0'; // 거짓
```

논리 연산자

진리값에 적용할 수 있는 연산자도 있다. 이를 논리 연산자라고 한다.

```
// ! 연산자는 논리적 NOT 연산자로 진리값을 역전시킨다.
// !참 => 거짓
// !거짓 => 참
!0; // 참
!1; // 거짓

// && 연산자는 논리적 AND 연산자로 피연산자의 진리값이 모두 참일 때 참이 된다.
// 참 && 참 => 참
// 참 && 거짓 => 거짓
// 거짓 && 참 => 거짓
// 거짓 && 거짓 => 거짓
1 && 1; // 참
```

```
1 && 0; // 거짓
0 && 1; // 거짓
0 && 0; // 거짓

// || 연산자는 논리적 OR 연산자로 피연산자의 진리값 중 하나라도 참이라면 참이다.
// 참 || 참 => 참
// 참 || 거짓 => 거짓
// 거짓 || 참 => 참
// 거짓 || 거짓 => 거짓
1 || 1; // 참
1 || 0; // 참
0 || 1; // 참
0 || 0; // 거짓
```

특히 && 연산자나 || 연산자의 경우 왼쪽 피연산자의 진리값이 참이냐 거짓이냐에 따라 오른쪽 피연산자의 진리값을 계산하지 않아도 최종 결과값을 알 수 있다. 이를 **단축 평가**(Short-Circuit Evaluation)이라 한다. 따라서 논리 연산자를 사용할 때는 단축 평가에 유의하여 좀 더 효율적으로 식을 작성하도록 하자.

```
0 && 1; // 오른쪽 식을 굳이 계산하지 않아도 거짓임을 알 수 있다.
        // 이때 오른쪽 식을 평가하지 않는다.

1 || 0; // 오른쪽 식을 굳이 계산하지 않아도 참임을 알 수 있다.
        // 이때 오른쪽 식을 평가하지 않는다.
```

또 드모르간의 법칙을 사용해 가독성 높게 식을 작성할 수도 있다.

```
// 로그인 기능을 구현하기 위해 아래 세 가지를 검증한다고 가정한다.
int isValidID;
int isValidPassword;
int isBrowserLatestVersion;

// 이를 조합해 로그인이 가능한지 판별하려 할 때,
// 조건식을 아래처럼 쓸 수 있다.
// 하지만 로그인 가능한 조건이 좀 더 까다로워지면 가독성을 해칠 수 있다.
!(isValidID && isValidPassword && isBrowserLatestVersion);

// 이 경우 드모르간의 법칙을 이용하면
// 똑같은 진리값을 얻더라도 좀 더 나은 가독성을 얻을 수 있다.
```

```
!isValidID || !isValidPassword || !isBrowserLatestVersion;
```

비교 연산자

비교 연산자는 비교 결과에 따라 진리값을 내놓는다.

```
// == 연산자는 두 값이 같은지 비교한다.
10 == 10; // 참
10 == 0; // 거짓

// != 연산자는 두 값이 다른지 비교한다.
10 != 10; // 거짓
10 != 0; // 참

// < 연산자는 왼쪽 피연산자가 오른쪽 피연산자보다 작은지 비교한다.
10 < 0; // 거짓
0 < 10; // 참

// > 연산자는 왼쪽 피연산자가 오른쪽 피연산자보다 큰지 비교한다.
10 > 0; // 참
0 > 10; // 거짓

// <= 연산자는 왼쪽 피연산자가 오른쪽 피연산자보다 작거나 같은지 비교한다.
10 <= 20; // 참
10 <= 10; // 참
10 <= 0; // 거짓

// >= 연산자는 왼쪽 피연산자가 오른쪽 피연산자보다 크거나 같은지 비교한다.
10 >= 20; // 거짓
10 >= 10; // 참
10 >= 0; // 참
```

if

if 문은 식의 진리값이 참일 때 특정 구문을 실행하게 할 수 있다.

```
#include <stdio.h>

int main(void)
{
    // if 문의 문법은 아래와 같다.
```

```
    // if (expression) statement
    if (1)
    {
        printf("참일 때 실행된다.\n");
    }

    return 0;
}
```

else절을 이용해 거짓일 때 특정 구문을 실행하게도 할 수 있다.

```
#include <stdio.h>

int main(void)
{
    if (0)
    {
        printf("참일 때 실행된다.\n");
    }
    // else 절은 가까운 if 문과 결합되며 다른 구문을 실행할 수 있다.
    else
    {
        printf("거짓일 때 실행된다.\n");
    }

    if (0);
    // 당연한 말이지만 구문이라는 것은 'if 문'도 포함된다.
    else if (0)
    {
        printf("여긴 실행이 되지 않는다.\n");
    }
    else
    {
        printf("여기가 실행되지롱\n");
    }

    return 0;
}
```

switch

switch 문도 if 문과 같이 여러 분기를 만들 수 있고, 특정 상황에서 if 문보다 switch 문으로 작성하는 것이 가독성과 편리함을 가져줄 때가 있다. 아래의 예시를 보자. 아래의 프로그램은 사용자로부터 하나의 알파벳 문자를 받아 그것이 자음인지 모음인지 판별하는 프로그램이다.

```
#include <stdio.h>

int main(void)
{
    char ch;
    scanf("%c", &ch);

    if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u'
|| ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O' || ch == 'U')
    {
        printf("모음입니다.\n");
    }
    else
    {
        printf("자음입니다.\n");
    }

    return 0;
}
```

조건식이 매우 길다는 것을 알 수 있다. if 문 대신 switch 문을 사용하면 아래와 같다.

```
#include <stdio.h>

int main(void)
{
    char ch;
    scanf("%c", &ch);

    switch (ch)
    {
    case 'a': case 'e': case 'i': case 'o': case 'u':
    case 'A': case 'E': case 'I': case 'O': case 'U':
        printf("모음입니다.\n");
        break;
```

```
    default:
        printf("자음입니다.\n");
        break;
    }

    return 0;
}
```

훨씬 더 깔끔함을 알 수 있다. switch 문의 흐름은 아래와 같다.

```
#include <stdio.h>

int main(void)
{
    int num;
    scanf("%d", &num);

    // switch문의 문법은 아래와 같다.
    // switch (expression) statement
    // 식의 타입은 반드시 정수 타입이어야 한다.
    switch (num % 10)
    {
    case 1: // :이 붙은 것을 레이블(Label)이라고 한다.
            // case 레이블은 case 정수형상수: 형식으로 적으며
            // 식의 평가값이 case와 같을 때를 의미한다.
            // (ex. num % 10 == 1)
        printf("나머지는 1입니다.\n");

        // break문을 적지 않으면 다음 case에 있는 구문이 이어서
        // 실행되게 된다.
        // 이를 fall-through 라고 한다.
    case 3:
        printf("나머지는 3입니다.\n");
        break; // break문을 만나면 더이상 아래의 구문이 실행되지 않는다.

    case 5:
        printf("나머지는 5입니다.\n");
        break;

    default: // default 레이블은 식의 평가값이
             // 어떤 case와도 일치하지 않을 때를 의미한다.
```

```
            // 생략할 수 있으나 보통은 적는 것을 권장한다.
            // 예외 처리에 사용되곤 한다.
        printf("자음입니다.\n");
        break;
    }

    return 0;
}
```

문제를 풀어보자.

반복문

기계는 결국 사람이 반복적인 일을 잘하지 못하고 지겨워하므로 탄생하게 되었다. 그런 의미에서 반복문이 있다는 것은 매우 자명한 일이다. 반복문은 조건이 참일 동안 특정 구문을 반복하는 구문이다.

while

while 문은 특정 조건이 참일 동안 구문을 반복한다. 반복 횟수를 정확히 모를 때 유용하다.

```
#include <stdio.h>

int main(void)
{
    // while 문의 문법은 아래와 같다.
    // while (expression) statement
    // expression의 진리값이 참일 동안 statement를 실행한다.

    int num = 10;
    while (num--)
    {
        printf("이 구문은 열 번 실행됩니다.\n");
    }

    // 조건식에 참값을 넣어 무한히 반복할 수도 있다.
    while (1)
    {
        printf("여기는 무한히 실행됩니다.\n");
    }
```

```
    return 0;
}
```

do-while

do-while 문은 반복 구문을 실행 후에 조건식을 평가한다. 조건식을 나중에 평가해야 하는 경우 유용하다.

```
#include <stdio.h>

int main(void)
{
    // do-while 문의 문법은 아래와 같다.
    // do statement while (expression);
    // statement를 실행 후 expression을 평가한다. expression이 참이라면
다시 statement를 실행한다.

    int num = 10;
    do
    {
        printf("이 구문은 열 번 실행됩니다.\n");
    } while (num--); // 뒤에 세미콜론을 빼먹지 않도록 주의한다.

    return 0;
}
```

for

for 문은 반복 횟수가 정해져 있을 때 유용하다.

```
#include <stdio.h>

int main(void)
{
    // for문의 문법은 아래와 같다.
    // for ( init-clause ; cond-expression ; iteration-expression )
    // loop-statement

    // init-clause는 for문을 만나면 가장 먼저 실행되며, 한 번만 실행된다.
    // 보통 선언을 하게 된다.
```

```
    // cond-expression은 루프문을 실행하기 전 평가되며,
    // 거짓일 시 for문이 종료된다.

    // iteration-expression은 루프문 실행 후에 평가되며,
    // 평가 후에는 cond-expression이 실행되게 된다.
    // 보통 반복 변수를 증가할 때 사용한다.

    // loop-statement은 반복할 구문이다.

    for (int i = 0; i < 10; ++i)
    {
        printf("이 구문은 열 번 실행됩니다.");
    }

    return 0;
}
```

break

break 문은 반복문을 중단시킬 수 있다.

```
#include <stdio.h>

int main(void)
{
    for (int i = 0; i < 10; ++i)
    {
        if (i == 5)
        {
            // i가 5가 됐을 때 for문을 종료한다.
            break;
        }

        printf("이 구문은 5번 실행됩니다.");
    }

    return 0;
}
```

continue

continue 문은 아래 구문의 실행을 생략하고 다음 루프로 진행시킨다.

```c
#include <stdio.h>

int main(void)
{
    for (int i = 1; i <= 10; ++i)
    {
        if (i % 2 == 0)
        {
            continue; // 이후 아래 있는 printf()문의 실행은 생략된다.
        }

        printf("이 구문은 i가 홀수일 때만 동작합니다.\n");
    }

    return 0;
}
```

다중 반복

반복문 안에 반복문을 사용하는 일도 흔하다. 문제는 다중 반복문에서 탈출하는 경우다.

```c
int main(void)
{
    for (int i = 0; i < 10; ++i)
    {
        for (int j = 0; j < 10; ++j)
        {
            if (i == 5 && j == 4)
            {
                // 반복문 전체를 멈추고 싶지만
                // break문과 continue문은 가장 가까운 반복문에만 적용된다.
                break;
            }
        }
    }

    return 0;
}
```

이 경우 변수를 사용하거나 goto 문을 사용할 수 있다.

```
int main(void)
{
    for (int i = 0; i < 10; ++i)
    {
        int canExit = 0;
        for (int j = 0; j < 10; ++j)
        {
            if (i == 5 && j == 4)
            {
                canExit = 1;

                break;
            }
        }

        if (canExit)
        {
            // 탈출 성공
            break;
        }
    }


    // 혹은

    for (int i = 0; i < 10; ++i)
    {
        for (int j = 0; j < 10; ++j)
        {
            if (i == 5 && j == 4)
            {
                goto LOOP_EXIT; // 레이블로 흐름을 이동시킨다.
            }
        }
    }

LOOP_EXIT: // 레이블은 Identifier: 형태로 작성한다.
           // 레이블은 함수 범위를 가지며,
           // 레이블 아래에는 하나 이상의 구문이 반드시 있어야 한다.

    return 0;
```

```
}
```

문제를 풀어보자.

포인터

포인터 타입은 다른 객체나 함수를 '가리킬 수 있는' 타입이다. 다시 말해 값을 메모리 주소값으로 해석한다. 정의하는 법은 아래와 같다.

```
// Pointer의 정의는 아래와 같다.
// type * identifier;
// 포인터 객체의 크기는 아키텍처에 따라 다르다.
// 32bit 아키텍처에서는 4바이트,
// 64bit 아키텍처에서는 8바이트이다.

int* p; // int 타입의 객체의 주소값을 저장할 수 있다.
        // p는 pointer to int라고 일컫는다.

int n = 10;

// 할당할 수 있는 데이터는 NULL이나 다른 객체의 주소값이다.
int* p2 = NULL; // 이런 포인터를 널 포인터(Null Pointer)라고 한다.
p2 = &n; // n의 주소값을 저장했다.
```

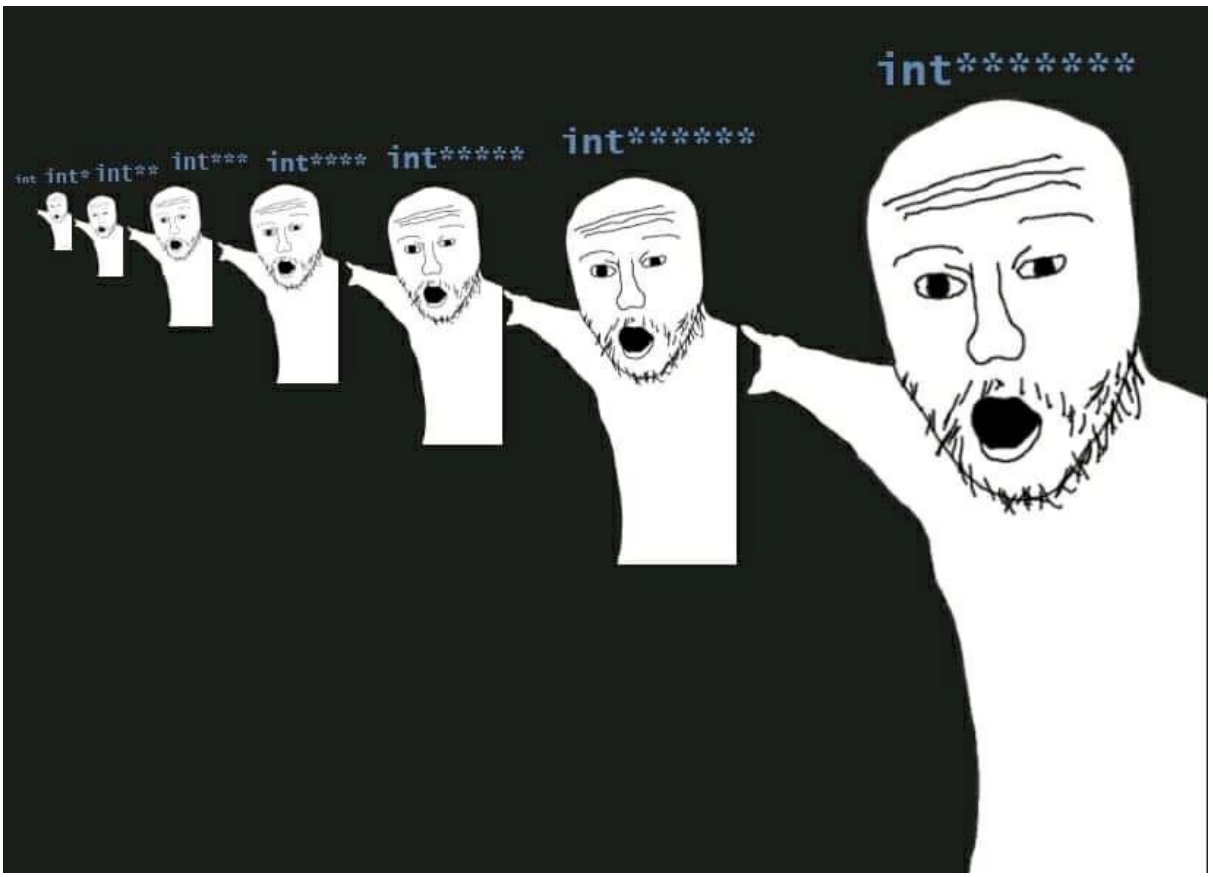
포인터를 사용해 접근하는 것을 **간접 참조**(Indirect Reference)*라고 한다.

*반대를 **직접 참조**(Direct Reference)라고 한다.

```
int n = 10;
int* p = &n;

*p = 20; // 간접 참조를 위해선
         // 역참조 연산자(Dereference Operator)를 사용한다.
         // 이때 n의 값은 20이다.
```

포인터 타입에 대해 잘 이해가 가지 않는다면 아래의 그림을 참고하자.



위 그림을 보다시피 포인터를 가리키는 포인터도 만들 수 있다.

```
int n = 10;
int* p = &n; // pointer to int
int** p2 = &p; // pointer to pointer to int
int*** p3 = &p2; // pointer to pointer to pointer to int

// 역참조를 할 때마다 가리키고 있는 객체로 이동한다고 생각하자.
*p = 20; // n의 값은 20
**p2 = 30; // n의 값은 30
***p3 = 40; // n의 값은 40

int a = 40;
*p2 = &a; // p는 이제 a를 가리키게 된다.
```

진리값

포인터 타입에도 진리값이 있다. 값이 NULL*이라면 거짓, 그 외의 값은 참이다.

*NULL의 실체가 0이라는 것을 생각하면 자명하다.

```
int* p = NULL;

if (p)
{
    // p가 NULL이므로 거짓이라 이 구문은 실행되지 않는다.
}
else
{
    // 이 구문이 실행된다.
}
```

**void*

포인터가 가리킬 수 있는 타입 중 void가 있다. void* 타입으로 정의하는 것은 데이터를 메모리 주소값 그 자체로 다룬다는 것을 의미한다. void* 타입은 포인터가 할 수 있는 연산 모두가 제한된다. 그 타입이 무엇인지 모르기 때문이다.

```
int n;
float f;
char ch;
const int c;

// void*는 어떤 타입이든 받을 수 있다.
void* p = &n;
p = &f;
p = &ch;
p = &c;

p = &n;
*p = 20; // 컴파일 오류
         // p는 현재 자신이 가리키고 있는 객체가 어떤 타입인지 알 수 없다.
         // 따라서 역참조를 한다는 것은 어불성설이다.
```

변환

포인터 타입도 변환이 존재한다. 우선 void* 타입은 다른 객체를 가리키는 포인터 타입으로 변환될 수 있다.

```
int n;
void* vp = &n;
```

```
int* ip = vp; // int* 타입으로 변환
*ip = 20; // n은 20

double* dp = vp; // double* 타입으로 변환
*dp = 3.2 // double 타입의 크기가 int 타입의 크기보다 크므로
          // 다른 메모리 침범하기 때문에 위험함.
          // 의도한 것과 다른 값이 할당됨.
```

정수 상수는 void* 타입으로 변환된다.

```
int* p = 0; // int 타입의 값 0이 void* 타입의 값 0이 된다.
            // void* 타입은 int* 타입으로 변환된다.

p = 0x1234; // 가능은 하나 무얼 가리키는 지는 알 수 없다.
```

왜 사용하는가?

절차지향 프로그래밍은 추후에 배울 함수라는 것으로 프로그램을 만들게 된다. 이 과정에서 자연스럽게 여러 가지 파일과 함수를 만들게 되는데, 그러다보면 다른 블록 범위에 있는 객체에 접근해야 할 때가 있다. 이 경우에 포인터를 사용해 다른 블록 범위에 있는 객체를 간접 참조하여 다루게 된다. 아직은 포인터가 갖는 가치가 와닿지 않을 수 있으나, 추후 함수를 배우면서 자연스럽게 알게 될 것이다. 현재는 포인터 타입이 가지는 값이 메모리 주소값임을 기억하자.

배열

프로그래밍을 하다 보면 비슷한 성질을 가지는 데이터를 여러 개 다뤄야 할 때가 있다. 가령 한 반에 30명인 학생의 점수를 가지고 순위를 매긴다고 해보자. 우리가 지금 할 수 있는 것은 변수를 30개를 만드는 것이다.

```
int grade1;
int grade2;
int grade3;
// ...
int grade30;
```

딱 봐도 뭔가 좋지 않다. 만일 학생 수가 30명이 아니라 100명이라면? 1000명이라면? 코드를 읽기도 매우 힘들어질 것이고, 작성하는 것도 쉽지 않을 것이다.

배열(Array)은 같은 타입으로 된 여러 개의 객체를 한 번에 다루고자 할 때 사용한다. 위의 예시를 배열로 바꿔보면 아래와 같다.

```
int grades[30];
```

데이터를 다루기가 한결 간편해졌음을 확인할 수 있다. 문법을 좀 더 자세히 살펴보자.

```
// 배열의 정의는 아래와 같다.
// type identifier[array-size];
// array-size는 정수 상수여야 한다.
// 배열의 크기는 type * array-size만큼이다.

int arr[30]; // arr는 int 타입의 객체를 30개 갖고 있는 객체다.
             // 객체 하나를 원소(Element)라 한다.
             // arr의 크기는 4 * 30 = 120 바이트다.

int arr2[5] = { 1, 2, 3, 4, 5 }; // 초기화는 {}을 사용한다.
                                 // 순서대로 각 원소에 데이터가 저장된다.
int arr3[5] = { 1, 2, 3 }; // 모든 원소에 대해 초기값을 지정하지 않을 시
                           // 자동으로 0이 된다.
int arr4[] = { 1, 2, 3 }; // 배열의 크기를 지정해주지 않고,
                          // 초기화를 했을 때는 자동으로 크기를 계산한다.
                          // 이 경우 arr4의 크기는 3이다.
```

메모리 상에서 배열은 각 원소가 연속적으로 배치된다. 위 예시에서 arr2를 그림으로 나타내면 다음과 같다.

데이터	1	2	3	4	5
메모리 주소	0x1234	0x1238	0x123c	0x1240	0x1244

원소에 접근하려면 [] 연산자를 사용하면 된다.

```
int arr[5] = { 1, 2, 3, 4, 5 };
arr[0]; // 배열의 첫 번째 원소. 값은 1
arr[1]; // 배열의 두 번째 원소. 값은 2
arr[2]; // 배열의 세 번째 원소. 값은 3
arr[3]; // 배열의 네 번째 원소. 값은 4
```

```
arr[4]; // 배열의 다섯 번째 원소. 값은 5
```

[] 연산자에 기입하는 숫자를 **인덱스**(Index)라고 하며, 0부터 시작함에 주의해야 한다. 이유는 후술한다.

다차원 배열

배열을 사용할 때 메모리를 논리적으로 구분하여 사용할 수 있다. 가령 학급 당 학생 수가 30명이고, 10개의 반이 있다고 해보자. 이를 배열로 나타내면 총 학생 수가 300명이므로 아래와 같이 정의할 수 있다.

```
int grades[300];
```

이 배열을 가지고 각 반의 특정 학생의 데이터에 접근하려면 약간의 계산식을 거쳐야 할 것이다.

```
int grades[300]; // 각 원소의 값은 적절하게 초기화가 되어 있다고 가정한다.
grades[2 * 30 + 2]; // 3반의 3번째 학생의 데이터에 접근했다.
```

보다시피 직관적이지 않다. 이런 경우 **다차원 배열**(Multi-dimentional Array)을 사용할 수 있다.

```
// 다차원 배열의 정의는 단순히 배열 선언자를 더 작성하면 된다.
// 늘릴 수 있는 차원 수는 메모리가 허락하는 한 무한히 늘릴 수 있다.

int arr1[5]; // 일차원 배열 (5)
int arr2[5][5]; // 2차원 배열 (5 x 5).
                // 2차원 배열은 마치 행렬처럼 생각할 수 있다.
int arr3[5][5][5]; // 3차원 배열 (5 x 5 x 5)

// 초기화는 {}을 더 써주면 된다.
int arr4[5][5] = {
    { 1, 2, 3, 4, 5 },
    { 6, 7, 8, 9, 10 }
};

int arr5[2][2][2] = {
    {
        { 1, 2 },
        { 3, 4 },
```

```
        { 5, 6 },
    },
    {
        { 1, 2 },
        { 3, 4 },
        { 5, 6 },
    },
    {
        { 1, 2 },
        { 3, 4 },
        { 5, 6 },
    }
};
```

다차원 배열을 이용해 위의 데이터를 나타내보면 아래와 같다.

```
int grades[10][30]; // 30명이 한 반이며 총 10개의 반으로 구성
grades[2][2]; // 3반의 3번째 학생의 데이터에 접근
```

포인터로의 변환

배열은 포인터로 암시적 변환이 일어난다.

```
int arr[5] = { 1, 2, 3, 4, 5 };
int* p = arr; // 포인터로 변환이 일어났다.
              // 이 경우 arr의 첫 번째 원소의 주소값이 저장된다.
*p = 20; // arr은 이제 { 20, 2, 3, 4, 5 }다.
```

다차원 배열도 마찬가지로 담을 수 있다.

```
int arr[5][5];
int arr2[5][5][5];

int** p = arr; // 2차원 배열은 이중 포인터를 사용한다.
int*** p2 = arr2; // 3차원 배열은 삼중 포인터를 사용한다.
```

배열 포인터

배열을 가리키는 포인터도 만들 수 있다.

```
// 배열을 가리키는 포인터를 정의할 땐 ()에 유의한다.
// 괄호를 사용하지 않으면 포인터 배열이 된다.
int (*parr)[5]; // pointer to array of int

int arr[5];
parr = &arr; // 가능

int arr2[4][5];
parr = arr2; // 가능

int arr3[4][4];
parr = arr3; // 가능하나 주소 연산 시 유효하지 않은 메모리에
             // 접근할 수 있으므로 사용에 유의
```

주소 연산

위에서 설명하지 않은 포인터 타입의 연산이 있다. 바로 **주소 연산**(Address Operation)이다. 주소 연산은 메모리 주소값에 대한 연산으로 증가와 감소 연산만 가능하다.

```
int arr[5] = { 1, 2, 3, 4, 5 };
int* p = arr; // p = &arr[0];

p = p + 2; // 메모리 주소값을 가리키고 있는 타입의 크기 * 숫자만큼 증가시킨다.
           // 현재 가리키고 있는 메모리는 &arr[2]이다.
*p = 10;   // arr은 이제 { 1, 2, 10, 4, 5 }

p = p - 1; // 메모리 주소값을 감소시킨다.
*p = 30;   // arr은 이제 { 1, 30, 10, 4, 5 }

// 증감 연산자 역시 사용 가능하다.
++p;
--p;
p++;
p--;

// 역참조 연산자와 결합 시 순서에 유의한다.

int n = *p++; // 역참조 후 주소를 증가한다.
              // p가 &arr[1]이었으므로 n의 값은 30
              // p는 현재 &arr[2]이다.
```

```
int n2 = *++p; // 주소 연산 후 역참조한다.
               // p가 &arr[2]이므로 주소 연산을 하면 &arr[3]
               // 역참조하면 n2의 값은 4
```

사실 [] 연산자도 주소 연산을 이용해 접근한다.

```
int arr[5];
arr[2]; // *(arr + 2);와 같다.

int arr2[5][5];
arr2[2][3]; // *(*(arr2 + 2) + 3); 와 같다.
```

배열을 다루는 법을 알았다면 여기서 문제를 풀어보자.

상수 포인터

포인터 타입도 const 타입의 수식을 받을 수 있다. 상수 포인터는 주소 연산이 불가능하다.

```
int arr[5] = { 1, 2, 3, 4, 5 };

// 상수 포인터는 * 뒤에 const를 붙이면 된다.
int * const p = arr; // const pointer to int
*p = 20; // 가리키고 있는 타입이 int이므로 역참조하여 값을 수정할 수 있다.
         // 이때 배열은 { 20, 2, 3, 4, 5 }

++p; // 컴파일 오류
     // const pointer이기 때문에 주소 연산이 불가능하다.

const int* p2 = arr; // pointer to const int
*p2 = 10; // 컴파일 오류
          // 가리키고 있는 타입이 const int 이므로 수정 불가능

++p2; // const pointer가 아니므로 주소 연산 가능
      // p2 = &arr[1]

const int * const p3 = arr; // const pointer to const int
*p3 = 20; // 컴파일 오류
++p3; // 컴파일 오류

int * const * const p4 = &p; // const pointer to
```

```
                                    // const pointer to int;
```

문자열

문자열은 char 타입의 배열을 의미한다. 프로그램은 문자열을 다룰 때 좀 특수하게 다루는데, 문자열의 끝은 널 문자(Null Character)로 판단한다.* 아래 예시를 참고하자.

* 이런 문자열을 널 종료 문자열(Null-terminated String)이라고 한다.

```
char str[] = "Hello"; // 문자열을 초기화할 땐 ""을 사용한다.
                      // 이때 str의 크기는 5가 아닌 6이다.
                      // o 뒤에 널 문자가 포함되어 있기 때문이다.
char str2[128] = ""; // 빈 문자열로도 초기화 할 수 있다.
                     // 이때 str2의 모든 원소의 값은 0이다. 즉, 널 문자다,

str = "World!"; // 초기화일 때만 문자열 리터럴을 사용할 수 있다.
                // 이 문법은 오류다.

// 이후 문자열을 다루는 것은 배열과 같다.
str[2] = 'o'; // 이제 str은 "Heolo"다.

char str[4] = "Hello"; // 문자열의 크기를 지정할 때,
                       // 초기화할 문자열의 크기보다 작으면
                       // 컴파일 오류가 발생한다.
```

우선 문자열과 관련된 문제를 풀어보며 문자열의 특성을 알아보도록 하자.

<string.h>

다뤄보면 알겠지만 문자열을 조작하는 것이 쉽진 않다. 그래서 C 언어에서는 문자열과 관련된 표준 라이브러리인 <string.h>를 제공한다. 여러가지가 있지만 이중 유용한 것을 꼽자면 아래와 같다.

- strcpy() / strcat() / strlen() / strcmp() / strchr() / strrchr() / strstr() / strtok()

문자열 함수를 다뤄보면 알겠지만 널 문자로 종료가 되지 않았을 때, 배열의 크기가 충분하지 않을 때 다른 메모리를 침범하는 것을 확인할 수 있다. 항상 2개를 잘 체크하도록 하자.

문자열 리터럴

문자열 리터럴은 Main()가 호출되기 전 데이터 영역 어딘가에 만들어두고 사용한다. 그래서 포인터로 문자열 리터럴을 가리킬 수도 있다.

```
const char* str = "Hello World!";
```

그래서 문자열을 초기화 한 다음 다시 저런 구문으로 재할당 할 수 없는 것이다.

```
char str[] = "Hello World!"; // 데이터 영역에 있는 문자열
                             // "Hello World!"를 str에 복사한다.

str = "Hello"; // str의 타입은 배열이고,
               // 문자열 리터럴의 타입은 const char*이다.
```

하지만 타입을 보면 알 수 있듯 문자열 리터럴을 조작할 순 없다. 애초에 수정 불가능한 영역에 문자열 리터럴이 만들어지기 때문이다.

```
char* str = "Hello World!"; // 설사 char*으로 가리킨다 한들
str[2] = 'o'; // 액세스 위반으로 프로그램이 종료된다.
```

열거형

기호 상수가 코드의 가독성에 도움을 준다는 것은 우리가 이전에 살펴본 바 있다.

```
// 플레이어의 이동 방향을 미리 설정한다.
const int LEFT_DIRECTION = 1;
const int RIGHT_DIRECTION = 2;
const int UP_DIRECTION = 3;
const int DOWN_DIRECTION = 4;

// 방향에 따라 이동을 처리한다.
int playerDirection; // 플레이어의 이동 방향을 저장하는 객체
switch (playerDirection)
{
case LEFT_DIRECTION:
    // 왼쪽으로 이동한다.
    break;
case RIGHT_DIRECTION:
    // 오른쪽으로 이동한다.
    break;
case UP_DIRECTION:
    // 위로 이동한다.
```

```
    break;
case DOWN_DIRECTION:
    // 아래로 이동한다.
    break;
default:
    printf("오류 발생\n");
    break;
}
```

기능이 좀 더 확장되어 기존 4방향에서 8방향의 이동이 가능하게 되었다.

```
// 플레이어의 이동 방향을 미리 설정한다.
const int LEFT_DIRECTION = 1;
const int RIGHT_DIRECTION = 2;
const int UP_DIRECTION = 3;
const int DOWN_DIRECTION = 4;

// 새로운 방향 지정
const int UP_LEFT_DIRECTION = 5;
const int UP_RIGHT_DIRECTION = 6;
const int DOWN_LEFT_DIRECTION = 7;
const int DOWN_RIGHT_DIRECTION = 8;

// 방향에 따라 이동을 처리한다.
int playerDirection; // 플레이어의 이동 방향을 저장하는 객체
switch (playerDirection)
{
case LEFT_DIRECTION:
    // 왼쪽으로 이동한다.
    break;
case RIGHT_DIRECTION:
    // 오른쪽으로 이동한다.
    break;
case UP_DIRECTION:
    // 위로 이동한다.
    break;
case DOWN_DIRECTION:
    // 아래로 이동한다.
    break;

// 새로운 방향에 대한 처리
```

```
case UP_LEFT_DIRECTION:
    // 왼쪽 위 대각선으로 이동한다.
    break;

// ... 이하 생략

default:
    printf("오류 발생\n");
    break;
}
```

새로운 방향에 대해 일일이 값을 지정하는 게 너무 불편하다. 이런 경우에 열거형(Enumeration)을 사용할 수 있다. 열거형은 연관 있는 기호 상수를 묶어서 다룰 수 있는 타입이다. 플레이어 이동 방향을 열거형으로 나타내면 아래와 같다.

```
// 열거형은 아래와 같이 사용한다.
// enum [identifier] { enumerator-list }
// 식별자는 생략 가능하다.

enum DIR
{
    // 열거형에 나열한 기호 상수는 모두 int 타입의 값을 가진다.
    DIR_LEFT,    // 첫 번째 값을 지정해주지 않을 시 0이다.
    DIR_RIGHT,   // 두 번째부터는 값을 지정하지 않을 시
                 // 이전 값의 + 1 이다.
    DIR_UP = 3,  // 명시적으로 값을 지정할 수도 있다.
    DIR_DOWN,
    DIR_UP_LEFT,
    DIR_UP_RIGHT,
    DIR_DOWN_LEFT,
    DIR_DOWN_RIGHT
};
```

타입이기 때문에 객체를 만드는 것도 가능하다.

```
// 식별자만 사용하면 C언어가 알아듣지 못한다.
// 꼭 앞에 enum을 명시해야 한다.
enum DIR playerDirection = DIR_LEFT; // 값은 열거형에 있는 기호 상수를
                                     // 이용해도 되고
```

```
playerDirection = 3; // 정수를 직접 지정할 수도 있다.
                     // 열거형과 int 타입 간에는 변환이 일어난다.
playerDirection = -1; // 열거형에 아예 없는 값을 지정할 수도 있다.
```

열거형의 크기는 int 타입과 같다.

```
sizeof enum DIR; // 4
```

구조체

지금까지 C언어가 제공하고 있는 여러 타입에 대해서 살펴봤다. 하지만 사실 C언어에서 제공하는 타입만으로는 현실 세계의 다양하고 복잡한 문제를 해결하기엔 어렵다. 그래서 우리는 문제를 해결하기 위해 여러 가지 타입을 조합하여 문제를 해결하기 위한 우리만의 새로운 타입을 만들어야 한다.* 구조체(Structure)는 여러 타입으로 구성된 새로운 타입을 만드는 데 사용한다.

* 이를 사용자 정의 타입(User-defined Type)이라고 한다.

```
// 구조체의 정의는 아래와 같다.
// struct [identifier] { struct-declaration-list }
struct Student
{
    int Age; // 구조체를 구성하는 타입을 멤버(Member)라고 한다.
    enum { A, B, O, AB } BloodType;
    char Name[24];
    struct Student* BestFriend; // 자신의 타입도 멤버로 사용할 수 있다.
};
```

메모리에는 아래와 같이 표현된다.

Age	BloodType	Name	BestFriend

객체를 만들고 초기화 하는 방법은 아래와 같다.

```
// 열거형과 마찬가지로 앞에 struct를 붙여야
// Student라는 식별자를 인식할 수 있다.
struct Student seonmun = { 20, A, "최선문", NULL };
// 멤버가 정의된 순서대로 값을 넣어주게 된다.
// Age = 20, BloodType = A, Name = "최선문", BestFriend = NULL
```

멤버에 접근하는 것은 . 연산자를 사용한다.

```
// 당연한 얘기지만 주소 연산을 활용한다.
seonmun.Age; // &seonmun에 접근하면 값을 얻을 수 있다. 값은 20
seonmun.BloodType; // &seonmun + sizeof(int)에 접근하면
                   // 값을 얻을 수 있다. 값은 B(1)
seonmun.Name; // &seonmun + sizeof(int) + sizeof(enum)에 접근하면
              // 값을 얻을 수 있다. 값은 "최선문"
seonmun.BestFriend; // &seonmun + sizeof(int) + sizeof(enum) +
                    // sizeof(Name)에 접근하면 값을 얻을 수 있다.
                    // 값은 NULL(0)
```

포인터를 이용해 접근할 때는 -> 연산자를 사용한다.

```
struct Student* p = &seonmun;

// 각 멤버에 접근하려면 아래와 같이 한다.
(*p).Age;

// 하지만 매번 위처럼 쓰는 것이 귀찮으므로
// 이를 축약한 형태가 -> 연산자이다.
p->Age;
```

패딩

구조체 크기는 일반적으로 멤버의 크기를 전부 더하면 된다.

```
struct Student
{
    int Age; // 크기는 4바이트
    enum { A, B, O, AB } BloodType; // 크기는 4바이트
    char Name[24]; // 크기는 24바이트
    struct Student* BestFriend; // 크기는 4바이트 혹은 8바이트
};
// 따라서 sizeof(struct Student)는 36 바이트 혹은 40 바이트
```

그럼 아래 구조체의 크기는 몇일까?

```
struct // 총 13바이트
{
    char ch;  // 1바이트
    double d; // 8바이트
    float f; // 4바이트
};
```

하지만 실제로 계산해보면 24바이트가 나오게 된다. 왜 그런 것일까? 멤버 접근 빈도를 줄이기 위해 패딩 바이트가 추가되었기 때문이다. 메모리 주소에 접근하려면 주소가 정렬되어 있어야 한다. 그렇지 않으면 데이터가 나눠져 저장될 수 있으므로 접근을 2번 해야 한다. 그래서 C 프로그램에서는 접근을 한 번에 하기 위해 패딩 바이트를 넣는다. 위의 경우 아래와 같이 패딩 바이트가 추가되어 있다.

```
struct
{
    char ch; // 1바이트 + 7바이트(패딩)
    double d; // 8바이트
    float f; // 4바이트 + 4바이트(패딩)
};
```

비트 필드

공용체

함수

함수(Function)는 복합 구문에 식별자를 붙인 C 언어 구성 요소다. 즉, 일련의 과정에 이름을 붙인 것이라고 할 수 있다. 함수는 헤더(Header)와 몸체(Body)로 나뉜다. 문법은 여기를 확인하자.

함수는 0개 이상의 매개변수(Parameter)를 가질 수 있으며, 매개변수는 함수의 몸체에서 활용될 수 있는 변수다. 매개변수는 호출 연산자를 사용할 때 인자(Argument)에 의해 초기화 된다. 또 return 구문을 사용해 호출자에게 결과값을 반환할 수 있다. 함수 안에서 함수를 정의할 순 없으며(단, 선언은 가능하다.), 그렇기에 함수는 파일 범위를 가진다.

가변 인자

하나의 함수로 다양한 인자를 받고 싶을 때, 가변 인자(Variadic Argument)를 사용할 수 있다. 가변 인자를 사용하고 싶다면 매개 변수 목록에 ...을 넣으면 되며, <stdarg.h> 라이브러리를 통해 가변 인자를 다룰 수 있다. 자세한 건 여기를 참고하자.

스택 프레임과 호출 규약

함수는 프로세스 주소 공간 중 스택을 이용한다. 다시 말해 함수 내부에서 선언된 변수는 모두 스택에 있다는 것이다.* 그럼 함수를 호출할 때 내부적으로 어떤 일이 일어날까? 함수가 끝났을 때 어떻게 실행 지점이 다시 호출자 쪽으로 돌아오게 되는 걸까? 바로 스택 프레임(Stack Frame)에 있다. 스택 프레임(Stack Frame)은 함수와 관련된 데이터를 말한다. 스택 프레임은 임시 데이터, 매개변수, 반환 주소로 구성되어 있다.

호출 규약(Calling Convention)은 함수가 어떻게 호출자로부터 인자를 받을 것이며, 결과값을 호출자에게 어떻게 돌려줄 것인지 규정해 놓은 것을 얘기한다. 호출 규약에 대한 내용은 여기서 확인하며, 이것은 컴파일러마다 구현한 내용이 다르므로 우리가 사용하는 MSVC에서의 호출 규약 내용은 여기를 확인한다. 꼭 알아야 하는 내용만 정리하면 아래와 같다.

- **__cdecl**

C, C++ 프로그램의 기본 호출 규약으로 오른쪽에서 왼쪽으로 인자를 스택에 넣는다. 스택 정리는 호출자가 하기에 가변 인자를 사용할 수 있으며, 이 때문에 __stdcall보다 실행파일 크기가 커지게 된다.

- **__stdcall**

Win32 API 함수에서 사용하는 호출 규약으로 오른쪽에서 왼쪽으로 인자를 스택에 넣는다. 스택 정리는 피호출된 함수가 한다. 따라서 가변 인자를 사용할 수 없기 때문에, 가변 함수는 __cdecl로 만들어지게 된다.

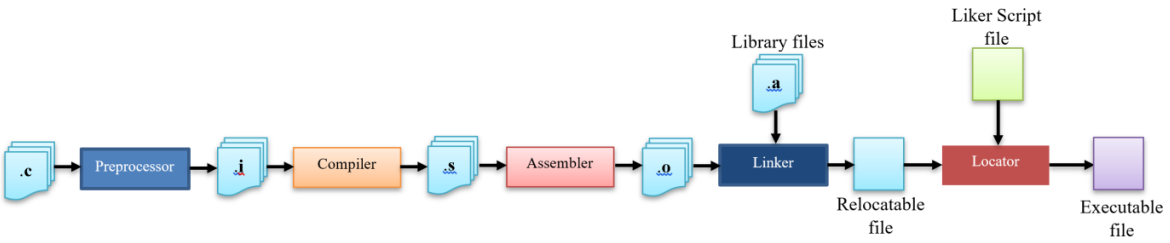
- **__fastcall**

첫 2개의 인자(크기가 4바이트보다 작아야 함)까지는 순서대로 ECX와 EDX 레지스터에 저장하고 나머지 인자는 오른쪽에서부터 왼쪽으로 스택에 넣는다. 스택 정리는 피호출된 함수가 한다.

위의 내용은 모두 x86 아키텍처 한정으로 x64 아키텍처 호출 규약은 또 다르다. 자세한 설명은 생략하며, 관심 있는 사람은 여기를 참고하자.

빌드

빌드(Build)는 코드로부터 프로그램을 만드는 과정을 의미한다. C 프로그램의 빌드 과정은 크게 전처리 - 컴파일 - 링크로 나뉘게 된다. 자세한 과정은 여기와 아래 이미지를 참고한다.



전처리

전처리(preprocessing)란 컴파일 전 일어나는 동작을 의미한다. 이를 위해 C언어에서는 다양한 전처리기를 지원하고 있다. 전처리기는 #으로 시작한다. 전처리로 할 수 있는 것을 하나씩 보자.

다른 파일 포함하기

#include는 다른 파일을 '포함'하는 전처리기다. 보통 헤더 파일(Header File)을 추가해 다른 소스 코드 파일에서 다른 파일에 있는 함수나 객체를 사용하기 위한 선언을 포함시키는 데 사용한다.

```
// 파일을 포함시킬 땐 2가지 방법이 있다.

// <>는 표준 라이브러리 혹은 써드파티 라이브러리를 포함하는 데 사용한다.
#include <stdio.h>

// ""는 내가(혹은 팀원이) 작성한 헤더 파일을 포함하는 데 사용한다.
#include "MyCode.h"
```

매크로 정의하기

#define은 매크로를 작성하는 데 사용한다. 매크로가 사용된 곳은 전처리 과정에 모두 '교체'된다.

```
#include <stdio.h>

// 매크로는 기호 상수처럼 사용할 수 있다.
#define SPEED_OF_LIGHT 299792458

// 함수처럼 사용할 수도 있다.
// 함수처럼 사용할 때는 매개변수에 꼭 ()를 사용하자.
```

```c
#define MIN(a, b) ((a) < (b) ? a : b)

// 매개변수에 #을 붙이면 문자열로 인식한다.
#define PRINT(msg) puts(#msg)

// 매개변수에 ##을 붙이면 문자끼리 잇는다.
#define DECLARE_MYTYPE(typename) struct MyType##typename

// 가변 인자도 사용할 수 있다.
// __VA_ARGS__는 ...이 들어갈 곳이다.
#define SIMPLE_PRINT(...) puts(#__VA_ARGS__)

#define FORMAT_PRINT(fmt, ...) printf(fmt, __VA_ARGS__)

int main(void)
{
    printf("빛의 속도는 %d m/s이다.\n", SPEED_OF_LIGHT); // 전처리 과정 때
SPEED_OF_LIGHT 매크로는 299792458로 대체된다.

    if (MIN(10, 20))
    {
        printf("10은 20보다 작다.\n");
    }

    PRINT(안녕하세요);

    DECLARE_MYTYPE(Student)
    {
        int a;
    } a;

    SIMPLE_PRINT(1, 'a', "Hello");

    FORMAT_PRINT("Hello Macro : %d, %c, %s", 12, 'c', "Hello");

    return 0;
}
```

#undef를 이용하면 정의했던 매크로를 해제할 수 있다.

```c
#include <stdio.h>
```

```
#define SPEED_OF_LIGHT 299792458
#undef SPEED_OF_LIGHT // 매크로 해제

int main(void)
{
    printf("빛의 속도는 %d m/s이다.\n", SPEED_OF_LIGHT); // 컴파일 오류

    return 0;
}
```

미리 정의된 매크로가 몇 개 있는데, 그 중 유용한 건 __LINE__ 과 __FILE__ 이다.

```
#include <stdio.h>

#define FILENAME "Test.c"

#define PRINT_ERROR(msg) printf("[%s:%d] %s\n", __FILE__, __LINE__,
msg)

int main(void)
{
    if (1)
    {
        PRINT_ERROR("오류가 발생했습니다.");
    }

#line 999 FILENAME // #line은 파일이름과 줄 번호를 바꾼다.
    if (1)
    {
        PRINT_ERROR("오류가 발생했습니다.");
    }

#line 20001 FILENAME
    if (1)
    {
        PRINT_ERROR("오류가 발생했습니다.");
    }

    return 0;
```

```
}
```

MSVC에서 미리 정의해 놓은 매크로도 있다.

조건부 컴파일

#if / #ifdef / #ifndef / #elif / #else / #endif 를 이용하면 식이나 매크로에 따라 다른 프로그램을 만들 수 있다.

```
#include <stdio.h>

int main(void)
{
    // 식이 참이라면 #endif까지의 내용을 포함한다.
#if 1
    puts("1. 이건 실행됨");
    // #if 류를 썼다면 반드시 #endif로 맺어야 한다.
#endif

// defined()를 이용해 매크로가 정의되었는지 체크할 수 있다.
#if defined(TEST)
    puts("2. 이건 실행 안됨");
    // #else로 #if의 식이 만족하지 않았을 때의 내용을 만들 수 있다.
#else
    puts("2. 이건 실행됨");
#endif

    // 앞에 !를 붙여 부정도 가능하다.
#if !defined(TEST)
    puts("3. 이건 실행됨");
#endif

    // #if defined를 #ifdef로 줄일 수 있다.
#define TEST
#ifdef TEST
    puts("4. 이건 실행됨");
#endif

    // 마찬가지로 #if !defined를 #ifndef로 줄일 수 있다.
#define T 10
#ifndef TEST
```

```
    puts("5. 이건 실행 안됨");
    // #elif는 else if 쓰는 것과 비슷하다.
#elif defined(T) && T > 2
    printf("5. 이건 실행됨");
#else
    printf("5. 이건 실행 안됨");
#endif

    return 0;
}
```

구현체 정의 동작

각 컴파일러마다 만들어 놓은 전처리기를 실행할 수도 있다. #pragma를 이용한다. MSVC의 전처리기는 여기서 볼 수 있다. 아래는 MSVC 전처리기 중 자주 사용되는 것이다.

- **#pragma once**

헤더 파일은 포함 순서에 따라 꼬일 수 있다.

```
// A.h
#include "B.h"

struct A { int n; };
// A.h End

// B.h
#include "A.h"

struct B { int n; };
// B.h End
```

위와 같은 상황이면 서로가 서로를 포함하게 돼 전처리 과정에서 수없이 많은 포함이 생기게 되고, 결국 빌드에 실패하게 된다. 이런 이유 때문에 헤더 인클루드 가드(Header Include Guard)라는 기법을 활용했다.

```
// A.h
#ifndef __A_H__
#define __A_H__
#include "B.h"
```

```
struct A { int n; };
#endif
// A.h End

// B.h
#ifndef __B_H__
#define __B_H__
#include "A.h"

struct B { int n; };
#endif
// B.h End
```

다만 매 헤더 파일마다 이런 작업을 하는 것이 쉽진 않다. 그래서 대다수의 컴파일러에서는 헤더 파일을 단 한 번만 포함시키는 #pragma once를 지원하고 있다.

```
// A.h
#pragma once
#include "B.h"

struct A { int n; };
// A.h End

// B.h
#pragma once
#include "A.h"

struct B { int n; };
// B.h End
```

- **#pragma pack**

    - **#include / #define**
    - 조건부 컴파일

컴파일

링크
연결이란 무엇인가.

    - **Linkage**

표준 라이브러리

<assert.h>

<ctype.h>

<float.h>

```
#include <assert.h>
#include <stdio.h>
#include <float.h>

int IsSame(float a, float b)
{
    if (a - b < FLT_EPSILON)
    {
        return 1;
    }
    else
    {
        return 0;
    }
}

int main(void)
{
    float f = 1.0 - 0.7;
    if (f == 0.3)
    {
        printf("출력 되니?");
    }

    if (IsSame(f, 0.3))
    {
        printf("같음");
    }
}
```

<limits.h>

<math.h>

<stdbool.h>

<stddef.h>

<stdint.h>

<time.h>

<stdlib.h>

```
#include <stdio.h>
#include <stdlib.h>

// 동적 할당
int main(void)
{
    // 10
    // int[10]
    printf("필요한 공간 개수를 입력해주세요. : ");
    int n;
    scanf_s("%d", &n);

    // 메모리는 자원(Resource)이다.
    // 자원이라는 것은 시스템으로부터 빌려와서 사용이 끝나면 돌려줘야 하는 것
    // 사용이 끝난 메모리를 돌려주지 않으면 메모리 누수(Memory Leak)가 발생

    int* p = malloc(sizeof(int) * n);
    for (int i = 0; i < 10; ++i)
    {
        p[i] = i;
    }

    free(p); // 힙에 메모리를 반환.
    p = NULL; // 해제가 됐다면 꼭 NULL 포인터로 만들어 줄 것

    // 메모리를 수동으로 관리할 때 문제점
```

```
    // 1. 언제 사용이 끝나는지 불명확함. => 반환이 안될 수 있음. => 메모리
누수 발생
    // 2. 반환이 됐음에도 불구하고 접근하는 경우 => 유효하지 않은 메모리 주소를
갖고 있는 포인터를 댕글링 포인터(Dangling Pointer) => 메모리 접근 위반
    // 3. 반환이 됐음에도 불구하고 또 반환하는 경우 => 이중 해제(Double Free)

    return 0;
}
```

<stdio.h>

```
#include <stdio.h>

// 파일 입출력
int main(void)
{
    // 1. 파일을 다루기 위한 객체 생성
    FILE* fp = NULL;

    // ------------------------------
    // 텍스트 파일
    // ------------------------------

    // 2. 파일을 연다.
    // 1) 텍스트 파일 : 텍스트 에디터로 열 수 있는 파일로 우리가 읽을 수 있는
문자로 구성
    // 2) 바이너리 파일 : 그 외의 모든 파일

    // Metaverse라는 이름을 가진 텍스트 파일을 연다.
    if (0 != fopen_s(&fp, "Metaverse", "w"))
    {
        printf("오류 발생함.");

        return 1;
    }

    // 3. 파일을 조작한다.
    // 텍스트 파일에 작성할 수 있는 함수 : fputs() / fputc() / fprintf()
    // 바이너리 파일에 작성할 수 있는 함수 : fwrite()
    fputs("Hello File!", fp);
```

```
    fputc('J', fp);
    fprintf(fp, "\nThe name of the queen : %s\n", "퀸균지");

    // 4. 파일을 닫는다.
    fclose(fp);

    if (0 != fopen_s(&fp, "Metaverse", "r"))
    {
        printf("오류 발생함.");

        return 1;
    }

    // 텍스트 파일에서 읽어들일 수 있는 함수 : fgets() / fgetc() / fscanf()
    // 바이너리 파일에서 읽어들일 수 있는 함수 : fread()
    char ch = fgetc(fp);
    char str[128] = "";
    fgets(str, sizeof(str), fp); // 개행 문자까지 받아들인다.
    char str2[128] = "";
    fscanf_s(fp, "The name of the queen : %s", str2, sizeof(str2));

    printf("읽어들인 문자 : %c\n", ch);
    printf("읽어들인 문자열 : %s\n", str);
    printf("퀸은 누구? : %s\n", str2);

    fclose(fp);

    // ------------------------------
    // 바이너리 파일
    // ------------------------------
    if (0 != fopen_s(&fp, "Metaverse2", "wb"))
    {
        printf("오류 발생함.");

        return 1;
    }

    struct Student
    {
        int Age;                        // 4바이트
        enum { A, B, O, AB } BloodType;
        char Name[24];
```

```
    };

    struct Student s = { 20, A, "최선문" };

    if (1 != fwrite(&s, sizeof(s), 1, fp))
    {
        printf("오류 발생함.");

        fclose(fp);

        return 1;
    }

    fclose(fp);

    if (0 != fopen_s(&fp, "Metaverse2", "rb"))
    {
        printf("오류 발생함.");

        fclose(fp);

        return 1;
    }

    struct Student s2 = { 0 };
    if (1 != fread(&s2, sizeof(s2), 1, fp))
    {
        printf("오류 발생함.");

        fclose(fp);

        return 1;
    }

    printf("나이 : %d, 혈액형 : %d, 이름 : %s\n", s2.Age, s2.BloodType,
s2.Name);

    fclose(fp);

    return 0;
}
```

```c
#include <stdio.h>

// 문자열 입출력
int main(void)
{
    const char* input = "이 시대의 진정한 퀸은 누구? 퀸균지";

    char str[128] = "";
    sscanf_s(input, "이 시대의 진정한 퀸은 누구? %s", str, (unsigned
int)sizeof str);

    char str2[128] = "";
    sprintf_s(str2, sizeof(str2), "그렇다. 퀸은 %s였다.", str, sizeof
str);

    puts(str2);

    return 0;
}
```

참고자료

- cppreference.com